

Lekce 15: Kurzory I - Čtení tabulek

Python pro GIS - Čtení atributové tabulky pomocí kurzoru

Vojtěch Barták, FŽP ČZU Praha

2025-12-03

Table of contents

1	Úvod	3
2	Čtení hodnot pomocí Search kurzoru	3
2.1	Typy kurzorů	3
2.2	Otevření kurzoru pomocí klauzule with	7
2.3	Speciální pole pro geometrii	7
2.4	Index kompaktnosti tvaru	10
2.5	Úlohy	11
3	Sumarizace tabulky pomocí kurzoru	11
3.1	Úlohy	13
4	Cyklus přes unikátní hodnoty pole	14
4.1	Získání unikátních hodnot z pole	14
4.2	Kombinace s výběrem prvků	15
4.3	Prostorové analýzy pro každou kategorii	16
4.4	Sumarizace a export podle kategorií	17
4.5	Podmíněné zpracování kategorií	18
4.6	Úlohy	19
5	Cyklus přes unikátní hodnoty rastru	20
5.1	Atributová tabulka rastru	20
5.2	ExtractByAttributes versus lokální algebra	22
5.3	Úlohy	23
6	Cheatsheet	24
6.1	Základy práce s kurzory	24
6.1.1	Vytvoření Search kurzoru	24

6.1.2	Použití s klauzulí with	24
6.1.3	Procházení kurzoru	24
6.2	Speciální pole pro geometrii	25
6.2.1	Základní geometrická pole	25
6.2.2	Práce s geometrickými poli	25
6.3	Sumarizace dat	26
6.3.1	Součet hodnot	26
6.3.2	Průměr	26
6.3.3	Maximum/minimum	26
6.3.4	Podmíněný součet	27
6.4	Práce s unikátními hodnotami	27
6.4.1	Získání unikátních hodnot pole	27
6.4.2	Procházení unikátních hodnot	28
6.4.3	Sumarizace podle kategorií	28
6.5	Práce s rastry	28
6.5.1	Čtení atributové tabulky rastru	28
6.5.2	Výpočet ploch kategorií	29
6.5.3	Extrakce kategorií rastru	29
6.6	SQL dotazy (WHERE clause)	29
6.6.1	Základní operátory	29
6.6.2	Použití v kurzoru	30
6.7	Časté vzory použití	30
6.7.1	Export prvků podle kategorie	30
6.7.2	Vytvoření slovníku z tabulky	30
6.7.3	Hledání maxima s atributy	31
6.7.4	Procházení s indexem	31
6.8	Tipy a triky	32
6.8.1	Rychlé získání počtu prvků	32
6.8.2	Práce s None hodnotami	32
6.8.3	Kombinace více WHERE podmínek	32
6.8.4	Zpracování s chybovým hlášením	32

1 Úvod

V této lekci si ukážeme, jak lze procházet atributové tabulky (či jiné tabulky ve formátu **.dbf**) řádek po řádku a číst hodnoty z jednotlivých polí (sloupců). Po absolvování lekce budete umět:

- Otevřít kurzorem **dbf** tabulku a číst hodnoty v jednotlivých sloupcích
- Číst geometrické vlastnosti jednotlivých prvků z pole **SHAPE**
- Provádět vlastní sumarizace tabulek
- Zpracovávat vektorová data “prvek po prvek”
- Procházet v cyklu unikátní hodnoty z nějakého pole
- Procházet unikátní hodnoty diskrétního rastru

💡 Prozkoumejte nejprve data!

Jednotlivé procedury si budeme zkoušet na [datech](#) krajů (polygony) a velkých měst (body) ČR. Než začnete, nahrajte si data do ArcGIS Pro a prohlédněte si atributové tabulky!

2 Čtení hodnot pomocí Search kurzoru

Procházení tabulky se děje pomocí tzv. *kurzoru*, což je objekt podobný tomu, jaký vytváříme při otevírání textových souborů funkcí **open**: zprostředkovává spojení s daným souborem (tabulkou), takže po celou dobu, kdy tabulku procházíme, je uzamčena pro úpravy jinými aplikacemi. Z toho důvodu je po ukončení práce s tabulkou nutné vytvořený kurzor smazat (příkazem **del**), podobně jako po ukončení práce s textovým souborem jej uzavřeme metodou **close**.

2.1 Typy kurzorů

Kurzory v **arcpy** jsou trojího druhu:

- **Search Cursor** - určený pro pouhé čtení hodnot z tabulky,
- **Update Cursor** - určený pro čtení a zápis do existujících řádků tabulky,
- **Insert Cursor** - určený pro vkládání nových řádků do tabulky a mazání existujících řádků.

V této lekci si představíme pouze **SearchCursor**. Zbylé kurzory probereme v následující lekci.

Syntaxe použití Search kurzoru je následující:

```
tab = arcpy.da.SearchCursor(in_table, field_names, {where_clause}, ...)
```

V uvedeném kódu ukládáme kurzor do proměnné `tab`. Část `arcpy.da` odkazuje na skutečnost, že kurzory jsou v balíčku `arcpy` implementovány v samostatném modulu `da` (zkratka pro “Data Access”). Kurzor se vytváří stejnojmenným konstruktorem dané třídy (funkce `SearchCursor` v ukázce). Nejdůležitějšími parametry funkce jsou:

- **in_table** - tabulka, která se má otevřít (celá cesta jako řetězec, případně relativní cesta vzhledem k pracovnímu adresáři) - může se jednat o shapefile, `.dbf` tabulku, diskretní rastr (který má tabulku), vrstvu (pokud jsme ji vytvořili či pokud pracujeme v Python Window v ArcMap) apod.,
- **field_names** - seznam polí (sloupců), která chceme otevřít (pythonovský seznam řetězců),
- **where_clause** - SQL dotaz (nepovinně), kterým můžeme omezit otevíranou tabulku pouze na vybrané řádky.

Další parametry

Kurzor obsahuje ještě další parametry (znázorněné v ukázce třemi tečkami), ty zde však probírat nebudeme. Za zmínku nicméně stojí parametry `spatial_filter` a `spatial_relationship`, pomocí nichž lze kurzorem otevřít pouze prvky vybrané prostorovým výběrem (tj. za základě daného prostorového vztahu k dané jiné vrstvě).

Kurzor je možné přímo procházet `for` cyklem. Iterační proměnná přitom prochází jednotlivé řádky a nabývá vždy hodnoty n -tice stejné délky, jako je délka řádku (tj. počet otevřených polí). Ukažme si to na příkladu otevření atributů “KATEGORIE” a “NAZEV” vrstvy sídel:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."
tab = arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV", "KATEGORIE"])
for r in tab:
    print(r)
del(tab)
```

Po spuštění skriptu obdržíme na konzoli následující výpis:

```
(u'Praha', 1)
(u'Brno', 2)
(u'Plze\u0148', 2)
(u'Ostrava', 2)
(u'Karlovy Vary', 3)
(u'Zl\u00e1na', 3)
```

```
(u'Chomutov', 3)
(u'Most', 3)
(u'\xdast\xed nad Labem', 3)
(u'Kladno', 3)
(u'Hradec Kr\xe1lov\xed', 3)
(u'Karvin\xed', 3)
(u'Pardubice', 3)
(u'Fr\xfddek-M\xedstek', 3)
(u'Hav\xed\u0159ov', 3)
(u'Liberec', 3)
(u'\u010cesk\xed Bud\u011bjovice', 3)
(u'Olomouc', 3)
```

Jeliko\u017e je \u0159\u00e1dek jednoduchou n -tic\u00ed, je mo\u017en\u00e9 k jednotliv\u00fdm polo\u017ek\u00e1m (sloupc\u00edm) \u0159\u00e1dku p\u0159istupovat pomocí index\u00fa:

```
# -*- coding: cp1250 -*-

import arcpy
arcpy.env.workspace = r"C:\my_path\..."
tab = arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV", "KATEGORIE"])
for r in tab:
    print(f"M\u011bsto kategorie {r[1]} se jmenuje {r[0]}.")
del(tab)
```

V\u00fdpis vypad\u00e1 n\u00e1sledovn\u011b:

```
M\u011bsto kategorie 1 se jmenuje Praha.
M\u011bsto kategorie 2 se jmenuje Brno.
M\u011bsto kategorie 2 se jmenuje Plze\u0148.
M\u011bsto kategorie 2 se jmenuje Ostrava.
M\u011bsto kategorie 3 se jmenuje Karlovy Vary.
M\u011bsto kategorie 3 se jmenuje Zl\u00edn.
M\u011bsto kategorie 3 se jmenuje Chomutov.
M\u011bsto kategorie 3 se jmenuje Most.
M\u011bsto kategorie 3 se jmenuje \u00dast\u00ed nad Labem.
M\u011bsto kategorie 3 se jmenuje Kladno.
M\u011bsto kategorie 3 se jmenuje Hradec Kr\u00e1lov\u00e9.
M\u011bsto kategorie 3 se jmenuje Karvin\u00e1.
M\u011bsto kategorie 3 se jmenuje Pardubice.
M\u011bsto kategorie 3 se jmenuje Fr\u00fddek-M\u00edstek.
M\u011bsto kategorie 3 se jmenuje Hav\u00ed\u0159ov.
```

```
Město kategorie 3 se jmenuje Liberec.  
Město kategorie 3 se jmenuje České Budějovice.  
Město kategorie 3 se jmenuje Olomouc.
```

Všimněme si, že jsme při otevírání tabulky změnili pořadí sloupců oproti tomu, jak se tabulka zobrazuje v mapovém dokumentu ArcGIS Pro. Při otevírání tabulky pomocí kurzoru můžeme zvolit libovolné sloupce v libovolném pořadí, dokonce můžeme jeden sloupec otevřít v několika kopiích (což má zřídka nějaký dobrý důvod). Při čtení hodnot z řádku je přitom pořadí položek přesně takové, v jakém pořadí jsme sloupce seřadili při vytváření kurzoru.

V předešlé ukázce jsme nevyužili volitelný parametr *where_clause*, kterým můžeme omezit otvírané řádky. Pokud bychom např. chtěli otevřít pouze záznamy o městech první a druhé kategorie, vypadal by kód takto:

```
import arcpy  
arcpy.env.workspace = r"C:\my_path\..."  
tab = arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV"], '"KATEGORIE" < 3')  
for r in tab:  
    print(r[0])  
del(tab)
```

Výsledek:

```
Praha  
Brno  
Plzeň  
Ostrava
```

Opakované procházení kurzorem

Po průchodu tabulkou `for` cyklem zůstává kurzor na konci tabulky a nelze jej již znovu procházet. K tomu účelu je nutné buď kurzor znovu vytvořit, nebo jej vrátit na začátek tabulky metodou `reset`:

```

# první procházení
tab = arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV"])
for r in tab:
    print(r[0])

# pokus o další procházení - NEFUNGUJE!
for r in tab:
    print(r[0])

# další procházení pomocí znovuvytvoření kurzoru
tab = arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV"])
for r in tab:
    print(r[0])

# další procházení pomocí metody reset
tab.reset()
for r in tab:
    print(r[0])

```

2.2 Otevření kurzoru pomocí klauzule with

Nyní si ukážeme, jak se lze vyhnout nutnosti kurzor po použití mazat, a to díky klauzuli `with`, kterou znáte (v podobném kontextu) z lekce o textových souborech:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."
with arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV"], '"KATEGORIE" < 3') as tab:
    for r in tab:
        print(r[0])

# Zde následují další příkazy, přičemž kurzor již je zavřený

```

2.3 Speciální pole pro geometrii

Kromě běžných atributových polí (jako "NAZEV", "POCET_OB" apod.) můžeme v kurzoru otevřít také speciální pole, která nám poskytují přístup ke geometrickým vlastnostem prvků. Tato pole mají speciální názvy začínající `SHAPE@`:

- `SHAPE@AREA` - plocha polygonu (v jednotkách souř. systému, typicky m²)

- SHAPE@LENGTH - délka linie nebo obvod polygonu (v jednotkách souř. systému, typicky m)
- SHAPE@XY - souřadnice bodu jako n -tice (x, y)
- SHAPE@TRUECENTROID - souřadnice těžiště polygonu jako n -tice (x, y)
- SHAPE@CENTROID - souřadnice těžiště polygonu, pokud těžiště leží uvnitř polygonu, jinak souřadnice popisku ("label point"), jako n -tice (x, y)

Ukažme si základní práci s těmito poli. Nejprve získáme plochy krajů:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Rozlohy krajů:")
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@AREA"]) as tab:
    for r in tab:
        nazev = r[0]
        plocha_m2 = r[1]
        plocha_km2 = plocha_m2 / 1000000 # převod na km²

        print(f"{nazev}: {round(plocha_km2, 2)} km²")
```

Můžeme kombinovat geometrická pole s běžnými atributy, například pro výpočet hustoty zalidnění:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Hustota zalidnění krajů:")
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB", "SHAPE@AREA"]) as tab:
    for r in tab:
        nazev = r[0]
        obyvatel = r[1]
        plocha_km2 = r[2] / 1000000

        hustota = obyvatel / plocha_km2

        print(f"{nazev}: {round(hustota, 2)} obyvatel/km²")
```

Pro body můžeme získat jejich souřadnice pomocí SHAPE@XY:


```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Souřadnice velkých měst:")
with arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV", "SHAPE@XY"]) as tab:
    for r in tab:
        nazev = r[0]
        x, y = r[1] # rozbalení n-tice souřadnic

        print(f"{nazev}: X = {x}, Y = {y}")

```

Všimněte si, že `SHAPE@XY` vrací *n*-tici (x, y), kterou můžeme ihned rozbalit do dvou proměnných.

Podobně můžeme získat souřadnice těžišť polygonů:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Těžiště krajů:")
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@TRUECENTROID"]) as tab:
    for r in tab:
        nazev = r[0]
        x, y = r[1]

        print(f"{nazev}: X = {x}, Y = {y}")

```

Pole `SHAPE@LENGTH` můžeme použít jak pro linie (kde vrací jejich délku), tak pro polygony (kde vrací obvod):

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Obvody krajů:")
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@LENGTH"]) as tab:
    for r in tab:
        nazev = r[0]
        obvod_m = r[1]
        obvod_km = obvod_m / 1000

        print(f"{nazev}: {round(obvod_km, 1)} km")

```

Můžeme také kombinovat více geometrických polí najednou, například pro výpočet indexu kompaktnosti tvaru:

```
import arcpy
import math
arcpy.env.workspace = r"C:\my_path\..."

print("Index kompaktnosti krajů (nižší = kompaktnější):")
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@AREA", "SHAPE@LENGTH"]) as tab:
    for r in tab:
        nazev = r[0]
        plocha = r[1] / 1000000 # km²
        obvod = r[2] / 1000 # km

        # Index kompaktnosti (kruh má nejnižší hodnotu 1)
        kompaktnost = (obvod ** 2) / (4 * math.pi * plocha)

        print(f"{nazev}: {round(kompaktnost, 2)}")
```

2.4 Index kompaktnosti tvaru

Index kompaktnosti porovnává tvar polygonu s kruhem (který má nejkompaktnější tvar ze všech útvarů). Vypočítá se jako:

$$\text{Kompaktnost} = \frac{O^2}{4\pi \cdot P}$$

kde O je obvod a P je plocha. Vzorec vychází z toho, že pro kruh platí $O = 2\pi r$ a $P = \pi r^2$. Po dosazení do vzorce dostaneme pro kruh hodnotu přesně 1. Čím více se tvar liší od kruhu (je protáhlejší, členitější), tím je hodnota vyšší. Například:

- Kruh: kompaktnost = 1,00
- Čtverec: kompaktnost 1,27
- Protáhlý obdélník: kompaktnost > 2,00

Konstantu π (pí) získáme v Pythonu importem modulu `math` a použitím `math.pi`.

Pokročilá práce s geometrií

Kromě zde uvedených polí existuje ještě `SHAPE@`, které vrací celý geometrický objekt s mnoha metodami pro pokročilé prostorové analýzy (buffery, průniky, obsahování atd.). Práci s těmito objekty si ukážeme v příští lekci.

2.5 Úlohy

Úloha 15.1. Otevřete tabulku krajů a vypište jejich názvy a počty obyvatel.

Úloha 15.2. Pomocí kurzoru najděte kraj s největší početní převahou žen nad muži.

Úloha 15.3. Pomocí geometrických polí vypočítejte a vypište hustotu zalidnění (obyvatel/km²) pro každý kraj. Najděte kraj s nejvyšší a nejnižší hustotou.

Úloha 15.4. Pro každé velké město vypište jeho název a souřadnice. Najděte město s nejvyšší souřadnicí Y (nejsevernější).

Úloha 15.5. Vypočítejte průměrnou plochu krajů v km². (Nápověda: projděte kurzorem všechny kraje, sečtěte plochy a vydělte počtem.)

Úloha 15.6. Pro jednotlivé plošky [krajinného pokryvu](#) vypočítejte index kompaktnosti tvaru. Jakého krajinného typu je ploška s nejkompaktnějším resp. nejméně kompaktním tvarem?

3 Sumarizace tabulky pomocí kurzoru

Jednou z velmi praktických aplikací kurzorů je provádění vlastních sumarizací tabulek. Zatímco nástroj **Statistics** nabízí předdefinovaný výběr statistik (suma, průměr, minimum, maximum apod.), pomocí kurzoru můžeme vypočítat jakoukoliv sumarizaci, kterou si sami naprogramujeme.

Princip je jednoduchý: procházíme tabulku kurzorem a průběžně akumulujeme hodnoty do proměnných. Ukažme si to na příkladu výpočtu celkového počtu obyvatel České republiky:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Inicializace sumy
celkem = 0

# Procházení tabulky
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        celkem += r[0]

print(f"Celkový počet obyvatel ČR: {celkem}")
```

Podobně můžeme vypočítat průměr, tentokrát s využitím čítače řádků:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Inicializace sumy a počítadla
suma = 0
pocet = 0

# Procházení tabulky
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        suma += r[0]
        pocet += 1

prumer = suma / pocet
print(f"Průměrný počet obyvatel na kraj: {prumer:.0f}")

```

Zajímavějším příkladem je výpočet, jaké procento rozlohy ČR tvoří kraje s hustotou zalidnění nad 150 ob/km²:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Inicializace
plocha_husté = 0 # plocha hustě zalidněných krajů
plocha_celkem = 0 # celková plocha

# Procházení tabulky
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB", "SHAPE@AREA"]) as tab:
    for r in tab:
        nazev = r[0]
        obyvatel = r[1]
        plocha_m2 = r[2]
        plocha_km2 = plocha_m2 / 1000000

        hustota = obyvatel / plocha_km2

        plocha_celkem += plocha_km2

        if hustota > 150:
            plocha_husté += plocha_km2
            print(f"{nazev}: {hustota:.1f} ob/km2")

```

```
procento = (plocha_husté / plocha_celkem) * 100
print(f"\nHustě zalidněné kraje (>150 ob/km2) tvoří {procento:.1f}% rozlohy ČR")
```

Podobně můžeme např. spočítat počet krajů s nadprůměrným počtem obyvatel:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Nejprve vypočítáme průměr
suma = 0
pocet = 0
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        suma += r[0]
        pocet += 1
prumer = suma / pocet

# Nyní spočítáme kraje nad průměrem
nad_prumerem = 0
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        if r[0] > prumer:
            nad_prumerem += 1

print(f"Počet krajů nad průměrem: {nad_prumerem} z {pocet}")
```

3.1 Úlohy

Úloha 15.7. Pomocí kurzoru vypočítejte celkový počet mužů a žen ve všech krajích ČR.

Úloha 15.8. Otevřete vrstvu okresů (*Okresy.shp*) a vypočítejte průměrnou hustotu zalidnění pro všechny okresy.

Úloha 15.9. Pro vrstvu Corine Land Cover (*CLC_CR.shp*) spočítejte celkovou plochu všech lesních ploch (kódy 311, 312, 313). Výsledek vypište v hektarech.

Úloha 15.10. Najděte tři okresy s nejvyšší hustotou zalidnění. Pro každý vypište název, počet obyvatel a hustotu. (Nápověda: použijte seznam pro ukládání dat a funkci `sorted()` pro seřazení.)

Úloha 15.11. Vypočítejte, kolik procent celkové rozlohy ČR zaujímají kraje s počtem obyvatel nad 1 milion.

Úloha 15.12. Pro vrstvu Corine Land Cover vytvořte CSV soubor s následujícími sloupci: CODE3 (kód kategorie), NAME3 (název kategorie), AREA_HA (plocha v hektarech), PERCENT (procento z celkové plochy). Soubor seřadte podle plochy sestupně. (Nápověda: nejprve projděte kurzorem všechny prvky a uložte data do seznamu slovníků, pak vypočítejte procenta a zapište do CSV.)

Úloha 15.13. Vytvořte CSV soubor se statistikami hustoty zalidnění pro všechny kraje ČR. Soubor by měl obsahovat sloupce: NAZEV, POCET_OB, ROZLOHA_KM2, HUSTOTA, KATEGORIE (kde kategorie je “velmi vysoká” pro hustotu >200, “vysoká” pro 100-200, “střední” pro 50-100, “nízká” pro <50).

Úloha 15.14. Pro každý okres spočítejte index kompaktnosti tvaru (podle vzorce z lekce) a najděte 5 nejkompaktnějších a 5 nejméně kompaktních okresů. Výsledky uložte do dvou samostatných CSV souborů s názvy `okresy_kompaktni.csv` a `okresy_nekompaktni.csv`.

4 Cyklus přes unikátní hodnoty pole

Častou úlohou při zpracování dat je procházet unikátní hodnoty nějakého kategoriálního pole a pro každou kategorii provádět specifickou operaci nebo GIS analýzu. Například můžeme chtít zpracovat všechny kraje zvlášť, analyzovat jednotlivé kategorie landuse, nebo provádět statistiky po okresech.

4.1 Získání unikátních hodnot z pole

K získání seznamu unikátních hodnot z pole použijeme `SearchCursor` v kombinaci s pythonovskou datovou strukturou `set` (množina), která automaticky odstraňuje duplicity:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Získání unikátních kategorií měst
kategorie = set()
with arcpy.da.SearchCursor("Sidla_body.shp", ["KATEGORIE"]) as tab:
    for r in tab:
        kategorie.add(r[0])

print(f"Kategorie měst: {sorted(kategorie)}")
```

Výstup:

Kategorie měst: [1, 2, 3]

Elegantnějším způsobem (pro pokročilé) je použití list comprehension s převodem na množinu:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Získání unikátních kategorií pomocí list comprehension
kategorie = set([r[0] for r in arcpy.da.SearchCursor("Sidla_body.shp", ["KATEGORIE"])]))
print(f"Kategorie měst: {sorted(kategorie)}")
```

💡 Množina (set) vs. seznam (list)

Datový typ `set` je vhodný pro unikátní hodnoty, protože:

- Automaticky odstraňuje duplicity
- Rychle testuje přítomnost prvku (`hodnota in mnozina`)
- Nepodporuje indexování, ale můžeme ho převést na seznam (`list(mnozina)`), nebo rovnou seřazený seznam (`sorted(mnozina)`)

4.2 Kombinace s výběrem prvků

Skutečná síla procházení unikátních hodnot spočívá v kombinaci s výběrem prvků pomocí `SelectLayerByAttribute`. To nám umožňuje provádět GIS analýzy samostatně pro každou kategorii.

Ukažme si základní příklad - pro každou kategorii měst vytvoříme buffer a vypočítáme jeho plochu:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Vytvoření vrstvy
lyr = arcpy.management.MakeFeatureLayer("Sidla_body.shp", "sidla_layer")

# Získání unikátních kategorií
kategorie = set()
with arcpy.da.SearchCursor("Sidla_body.shp", ["KATEGORIE"]) as tab:
    for r in tab:
        kategorie.add(r[0])

print("Analýza bufferů podle kategorií měst:")
for kat in sorted(kategorie):
```

```

# Výběr měst dané kategorie
where = f'"KATEGORIE" = {kat}'
arcpy.management.SelectLayerByAttribute(lyr, "NEW_SELECTION", where)

# Vytvoření bufferu pro vybraná města
buffer_output = f"buffer_kategorie_{kat}.shp"
arcpy.analysis.Buffer(lyr, buffer_output, "10000") # 10 km

# Vypočítání celkové plochy bufferů
celkova_plocha = 0
with arcpy.da.SearchCursor(buffer_output, ["SHAPE@AREA"]) as tab:
    for r in tab:
        celkova_plocha += r[0] / 1000000 # převod na km²

print(f"Kategorie {kat}: celková plocha bufferů = {round(celkova_plocha, 2)} km²")

# Zrušení výběru
arcpy.SelectLayerByAttribute_management("sidla_layer", "CLEAR_SELECTION")

```

4.3 Prostorové analýzy pro každou kategorii

Mocným nástrojem je provádění prostorových analýz pro jednotlivé kategorie. Například můžeme analyzovat, kolik lesních ploch leží v jednotlivých krajích:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Vytvoření vrstev
arcpy.MakeFeatureLayer_management("Kraje.shp", "kraje_layer")
arcpy.MakeFeatureLayer_management("CLC_CR.shp", "clc_layer")

# Získání seznamu krajů
kraje = set()
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV"]) as tab:
    for r in tab:
        kraje.add(r[0])

print("Plocha lesů v jednotlivých krajích:")
for kraj in sorted(kraje):
    # Výběr kraje

```



```

where_kraj = f'"NAZEV" = \'{kraj}\''
arcpy.management.SelectLayerByAttribute("kraje_layer", "NEW_SELECTION", where_kraj)

# Výběr lesních ploch (kódy 311, 312, 313)
where_les = '"CODE3" IN (311, 312, 313)'
arcpy.management.SelectLayerByAttribute("clc_layer", "NEW_SELECTION", where_les)

# Průnik - lesy v daném kraji
prunik = f"lesy_{kraj.replace(' ', '_')}.shp"
arcpy.analysis.Clip("clc_layer", "kraje_layer", prunik)

# Výpočet celkové plochy lesů
plocha_lesy = 0
with arcpy.da.SearchCursor(prunik, ["SHAPE@AREA"]) as tab:
    for r in tab:
        plocha_lesy += r[0] / 10000 # převod na ha

print(f"{kraj}: {round(plocha_lesy, 1)} ha lesů")

```

4.4 Sumarizace a export podle kategorií

Často potřebujeme vytvořit samostatné výstupy nebo statistiky pro každou kategorii:

```

import arcpy
import csv
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Získání unikátních krajů
kraje = set()
with arcpy.da.SearchCursor("Okresy.shp", ["KRAJ"]) as tab:
    for r in tab:
        kraje.add(r[0])

# Příprava CSV souboru
with open("statistiky_kraje.csv", "w", encoding="utf-8") as f:
    f.write("KRAJ;POCET_OKRESU;CELKOVA_PLOCHA_KM2;PRUMERNA_HUSTOTA\n"]

    for kraj in sorted(kraje):

        # Sumarizace pomocí kurzoru

```

```

pocet_okresu = 0
celkova_plocha = 0
celkem_obyvatel = 0

# SQL pro výběr okresů v daném kraji
where = f'"KRAJ" = \'{kraj}\''

# Otevření tabulky kurzorem a současný výběr okresů v daném kraji
with arcpy.da.SearchCursor("okresy_layer", ["POCET_OB", "SHAPE@AREA"], where) as tab:
    for r in tab:
        pocet_okresu += 1
        celkem_obyvatel += r[0]
        celkova_plocha += r[1] / 1000000 # km2

if celkova_plocha > 0:
    prumerna_hustota = celkem_obyvatel / celkova_plocha
else:
    prumerna_hustota = 0

# Zápis do CSV
f.write(f'{kraj};{pocet_okresu};{round(celkova_plocha, 2)};{round(prumerna_hustota, 2)};')

print(f'{kraj}: {pocet_okresu} okresů, {celkova_plocha:.0f} km2, hustota {prumerna_hustota:.0f} ob/km2')

print("\nStatistiky exportovány do: statistiky_kraje.csv")

```

4.5 Podmíněné zpracování kategorií

Můžeme také zpracovávat pouze vybrané kategorie podle nějakého kritéria:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Vytvoření vrstvy
arcpy.MakeFeatureLayer_management("Kraje.shp", "kraje_layer")

# Získání seznamu krajů
kraje = set([r[0] for r in arcpy.da.SearchCursor("Kraje.shp", ["NAZEV"])]))

print("Buffery pro velké kraje (>1 mil. obyvatel):")

```

```

for kraj in sorted(kraje):
    # Zjištění počtu obyvatel
    where = f'"NAZEV" = \'{kraj}\''
    with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"], where) as tab:
        pocet_ob = next(tab)[0]

    # Zpracování pouze velkých krajů
    if pocet_ob > 1000000:
        # Výběr kraje
        arcpy.SelectLayerByAttribute_management("kraje_layer", "NEW_SELECTION", where)

        # Vytvoření bufferu
        buffer_output = f"buffer_{kraj.replace(' ', '_')}.shp"
        arcpy.Buffer_analysis("kraje_layer", buffer_output, "5000") # 5 km

        print(f" {kraj} ({pocet_ob:,} ob.) - buffer vytvořen")

```

💡 Vymazání vrstvy po použití

Vrstvy vytvořené pomocí **MakeFeatureLayer** existují v paměti pouze během běhu skriptu. Pokud je chcete explicitně smazat, použijte:

```
arcpy.Delete_management("nazev_vrstvy")
```

4.6 Úlohy

Úloha 15.15. Pro každou kategorii Corine Land Cover (pole CODE3) vytvořte samostatný shapefile a vypište, kolik ploch každá kategorie obsahuje.

Úloha 15.16. Pro každý kraj vypočítejte, kolik procent jeho plochy zaujímají zastavěné plochy (CLC kódy začínající na 1). Výsledky exportujte do CSV souboru.

Úloha 15.17. Pro každou kategorii měst (1, 2, 3) vytvořte buffer 20 km a spočítejte, kolik obyvatel žije v krajích, které se s tímto bufferem překrývají. (Nápověda: použijte **Intersect** mezi bufferem a kraji, pak sečtěte obyvatele.)

Úloha 15.18. Pro každý okres najděte nejbližší velké město (kategorie 1 nebo 2). Výsledky uložte do CSV s sloupci: OKRES, NEJBЛИZSI_MESTO, VZDALENOST_KM. (Nápověda: použijte nástroj **Near** nebo vlastní výpočet vzdáleností pomocí těžišť.)

5 Cyklus přes unikátní hodnoty rastru

Podobně jako u vektorových dat často potřebujeme zpracovávat diskrétní rastry podle jejich unikátních hodnot. Typickým příkladem je landuse/landcover rastr, kde každá hodnota představuje jiný typ pokryvu.

Diskrétní rastr má v ArcGIS automaticky vytvořenou atributovou tabulku, kterou můžeme otevřít kurzorem stejně jako tabulku vektorové vrstvy. Ukažme si to na příkladu:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Otevření atributové tabulky rastru
with arcpy.da.SearchCursor("landuse.tif", ["VALUE", "COUNT"]) as tab:
    for r in tab:
        hodnota = r[0]
        pocet_pixelu = r[1]
        print(f"Hodnota {hodnota}: {pocet_pixelu} pixelů")
```

5.1 Atributová tabulka rastru

Atributová tabulka diskrétního rastru obsahuje standardně tato pole:

- VALUE - hodnota pixelu (kategorie)
- COUNT - počet pixelů s danou hodnotou
- případně další pole přidaná uživatelem

Pokud rastr náhodou nemá atributovou tabulku, musíte ji nejprve vytvořit nástrojem BuildRasterAttributeTable.

Získání unikátních hodnot z rastru pomocí kurzoru:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Získání všech hodnot z rastru
hodnoty = set([r[0] for r in arcpy.da.SearchCursor("landuse.tif", ["Value"])])
print(f"Unikátní hodnoty v rastru: {sorted(hodnoty)}")
```

Nyní můžeme pro každou hodnotu provádět samostatné analýzy. Například můžeme vytvořit samostatný rastr pro každou kategorii:

```

import arcpy

arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True
arcpy.CheckOutExtension("Spatial") # Aktivace rozšíření Spatial Analyst

# Vstupní rastrový soubor
in_raster = "landuse.tif"

# Získání unikátních hodnot v rastrovém souboru
vals = set()
with arcpy.da.SearchCursor(in_raster, ["Value"]) as cursor:
    for row in cursor:
        vals.add(row[0])

# Vytvoření binárních rastrů pro každou unikátní hodnotu
for v in sorted(vals):

    # Vytvoření objektu Raster
    ras = arcpy.sa.Raster(in_raster)

    # Vytvoření binárního rastru
    out_ras = ras == v

    # Uložení výsledného rastru
    out_ras.save(f"CLC_2018_{v}.tif")

```

Můžeme také spočítat plochu jednotlivých kategorií (pokud známe velikost pixelu):

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Získání velikosti pixelu
popis = arcpy.Describe("landuse.tif")
velikost_pixelu = popis.meanCellWidth * popis.meanCellHeight # v m2

print("Plochy jednotlivých kategorií:")
with arcpy.da.SearchCursor("landuse.tif", ["VALUE", "COUNT"]) as tab:
    for r in tab:
        hodnota = r[0]
        pocet = r[1]
        plocha_ha = (pocet * velikost_pixelu) / 10000 # převod na ha

```

```
print(f"Kategorie {hodnota}: {plocha_ha:.2f} ha")
```

Praktickým příkladem je vytvoření vektorové vrstvy polygon pro každou kategorii rastru:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Získání unikátních hodnot
hodnoty = set()
with arcpy.da.SearchCursor("landuse.tif", ["VALUE"]) as cursor:
    for row in cursor:
        hodnoty.add(row[0])

# Konverze každé kategorie na polygony
for hodnota in sorted(hodnoty):
    # Extrakce dané kategorie
    vyraz = f'"VALUE" = {hodnota}'
    rastr_kategorie = arcpy.sa.ExtractByAttributes("landuse.tif", vyraz)

    # Konverze na polygony
    vystup = f"landuse_poly_{hodnota}.shp"
    arcpy.conversion.RasterToPolygon(rastr_kategorie, vystup, "NO_SIMPLIFY")

    print(f"Vytvořena polygonová vrstva: {vystup}")
```

5.2 ExtractByAttributes versus lokální algebra

Může vzniknout otázka, proč jsme v této ukázce použili nástroj ExtractByAttributes namísto lokální mapové algebry na způsob přechodích ukázek. Důvod je ten, že zatímco z lokální algebry (porovnáním pomocí ==) dostaneme rastr s hodnotami 0/1, ExtractByAttributes dá buňkám mimo vybranou třídu hodnotu NoData. Při následném konverzi rastru na polygony se tudíž převedou pouze buňky reprezentující danou třídu.

Pokročilejší příklad kombinuje procházení kategorií rastru s prostorovými analýzami:

```
import arcpy
from arcpy.sa import *
arcpy.env.workspace = r"C:\my_path\..."

# Načtení referenční vrstvy (např. chráněná území)
```

```

chranena_uzemi = "chranena_uzemi.shp"

# Získání unikátních hodnot landuse
hodnoty = set()
with arcpy.da.SearchCursor("landuse.tif", ["VALUE"]) as cursor:
    for row in cursor:
        hodnoty.add(row[0])

print("Zastoupení kategorií v chráněných územích:")
for hodnota in sorted(hodnoty):
    # Extrakce dané kategorie
    rastr_kategorie = arcpy.sa.ExtractByAttributes("landuse.tif", f'"VALUE" = {hodnota}')

    # Oříznutí na chráněná území
    rastr_clip = arcpy.sa.ExtractByMask(rastr_kategorie, chranena_uzemi)

    # Spočítání buněk
    pocet = 0
    pocet += r[0]
    with arcpy.da.SearchCursor(rastr_clip, ["COUNT"]) as tab:
        for r in tab:
            pocet += r[0]

    if pocet > 0:
        print(f"Kategorie {hodnota}: {pocet} buněk v CHÚ")

```

Práce s velkými rastry

Při zpracování velkých rastrů může být operace s kurzory pomalá. V takových případech zvažte:

- Použití nástroje `ZonalStatisticsAsTable` pro sumarizace
- Práci s pyramid levels pro rychlejší přístup
- Zpracování po částech (tiles)

5.3 Úlohy

Úloha 15.19. Pro diskrétní rastr landuse vypište všechny kategorie a jejich procentuální zastoupení vzhledem k celkové ploše.

Úloha 15.20. Najděte kategorii, která má největší průměrnou výšku (použijte výškový rastr DEM a nástroj `ZonalStatisticsAsTable` nebo vlastní analýzu s kurzory).

Úloha 15.21. Pro každou kategorii landuse vytvořte buffer 100 m a spočítejte, jaká část původní kategorie zůstává uvnitř tohoto bufferu. (Nápověda: nejprve převedte rastr na polygony, vytvořte buffer a použijte `Clip` nebo prostorové operace.)

6 Cheatsheet

6.1 Základy práce s kurzory

6.1.1 Vytvoření Search kurzoru

```
# Základní syntaxe
tab = arcpy.da.SearchCursor(in_table, field_names, where_clause)

# Příklad s konkrétními parametry
tab = arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB"])

# S SQL dotazem
tab = arcpy.da.SearchCursor("Sidla.shp", ["NAZEV"], '"KATEGORIE" < 3')

# Uzavření kurzoru
del(tab)
```

6.1.2 Použití s klauzulí with

```
# Doporučený způsob - automatické uzavření
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV"]) as tab:
    for r in tab:
        print(r[0])
```

6.1.3 Procházení kurzoru


```
# For cyklus (doporučeno)
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as tab:
    for r in tab:
        print(f"{r[0]}: {r[1]} obyvatel")

# Metoda next() - posun na další řádek
tab = arcpy.da.SearchCursor("Kraje.shp", ["NAZEV"])
prvni_radek = tab.next()
druhy_radek = tab.next()
del(tab)
```

6.2 Speciální pole pro geometrii

6.2.1 Základní geometrická pole

```
# Souřadnice bodu
["SHAPE@XY"] # vrací (x, y) tuple

# Plocha polygonu
["SHAPE@AREA"] # v jednotkách souř. systému (typicky m²)

# Délka linie nebo obvod polygonu
["SHAPE@LENGTH"] # v jednotkách souř. systému (typicky m)

# Těžiště polygonu
["SHAPE@TRUECENTROID"] # vrací (x, y) tuple
```

6.2.2 Práce s geometrickými poli

```
# Výpočet kompaktnosti z plochy a obvodu
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@AREA", "SHAPE@LENGTH"]) as tab:
    for r in tab:
        nazev = r[0]
        plocha = r[1] / 1000000 # převod na km²
        obvod = r[2] / 1000 # převod na km
        kompaktnost = (obvod ** 2) / plocha
        print(f"{nazev}: {kompaktnost:.2f}")
```

```
# Získání souřadnic bodů
with arcpy.da.SearchCursor("Sidla.shp", ["NAZEV", "SHAPE@XY"]) as tab:
    for r in tab:
        nazev = r[0]
        x, y = r[1] # rozbalení tuple
        print(f"{nazev}: X={x:.0f}, Y={y:.0f}")

# Těžiště polygonů
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@TRUECENTROID"]) as tab:
    for r in tab:
        nazev = r[0]
        x, y = r[1]
        print(f"Těžiště {nazev}: X={x:.0f}, Y={y:.0f}")
```

6.3 Sumarizace dat

6.3.1 Součet hodnot

```
celkem = 0
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        celkem += r[0]
print(f"Celkem: {celkem}")
```

6.3.2 Průměr

```
suma = 0
pocet = 0
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        suma += r[0]
        pocet += 1
prumer = suma / pocet
```

6.3.3 Maximum/minimum

```

maximum = float('-inf')
nazev_max = ""

with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as tab:
    for r in tab:
        if r[1] > maximum:
            maximum = r[1]
            nazev_max = r[0]

print(f"Největší: {nazev_max} ({maximum} ob.)")

```

6.3.4 Podmíněný součet

```

suma_velkych = 0
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        if r[0] > 1000000:
            suma_velkych += r[0]

```

6.4 Práce s unikátními hodnotami

6.4.1 Získání unikátních hodnot pole

```

# Z vektorové vrstvy pomocí SearchCursor a set
hodnoty = set([r[0] for r in arcpy.da.SearchCursor("Sidla.shp", ["KATEGORIE"])]))

# Alternativně s explicitním cyklem
hodnoty = set()
with arcpy.da.SearchCursor("Sidla.shp", ["KATEGORIE"]) as tab:
    for r in tab:
        hodnoty.add(r[0])

# Z rastru (atributová tabulka)
hodnoty = set([r[0] for r in arcpy.da.SearchCursor("landuse.tif", ["VALUE"])]))

```

6.4.2 Procházení unikátních hodnot

```
kategorie = set([r[0] for r in arcpy.da.SearchCursor("Sidla.shp", ["KATEGORIE"])]))

for kat in sorted(kategorie):
    where = f'"KATEGORIE" = {kat}'
    with arcpy.da.SearchCursor("Sidla.shp", ["NAZEV"], where) as tab:
        for r in tab:
            print(f"Kategorie {kat}: {r[0]}")
```

6.4.3 Sumarizace podle kategorií

```
kategorie = set([r[0] for r in arcpy.da.SearchCursor("Sidla.shp", ["KATEGORIE"])]))
statistiky = {}

for kat in kategorie:
    where = f'"KATEGORIE" = {kat}'
    pocet = 0

    with arcpy.da.SearchCursor("Sidla.shp", ["NAZEV"], where) as tab:
        for r in tab:
            pocet += 1

    statistiky[kat] = pocet

print(statistiky)
```

6.5 Práce s rastry

6.5.1 Čtení atributové tabulky rastru

```
with arcpy.da.SearchCursor("landuse.tif", ["VALUE", "COUNT"]) as tab:
    for r in tab:
        hodnota = r[0]
        pocet_pixelu = r[1]
        print(f"Hodnota {hodnota}: {pocet_pixelu} px")
```

6.5.2 Výpočet ploch kategorií

```
popis = arcpy.Describe("landuse.tif")
velikost_px = popis.meanCellWidth * popis.meanCellHeight

with arcpy.da.SearchCursor("landuse.tif", ["VALUE", "COUNT"]) as tab:
    for r in tab:
        plocha_ha = (r[1] * velikost_px) / 10000
        print(f"Kategorie {r[0]}: {plocha_ha:.2f} ha")
```

6.5.3 Extrakce kategorií rastru

```
hodnoty = set([r[0] for r in arcpy.da.SearchCursor("landuse.tif", ["VALUE"])])

for hodnota in hodnoty:
    vyraz = f'"VALUE" = {hodnota}'
    rastr_kat = arcpy.sa.ExtractByAttributes("landuse.tif", vyraz)
    rastr_kat.save(f"kategorie_{hodnota}.tif")
```

6.6 SQL dotazy (WHERE clause)

6.6.1 Základní operátory

```
# Rovnost
'"KATEGORIE" = 1'
'"NAZEV" = \'Praha\''

# Nerovnost
'"KATEGORIE" <> 1'

# Porovnání
'"POCET_OB" > 1000000'
'"POCET_OB" <= 500000'

# Logické operátory
'"KATEGORIE" = 1 OR "KATEGORIE" = 2'
'"POCET_OB" > 100000 AND "POCET_OB" < 1000000'
```

```
# IN operator
'"KATEGORIE" IN (1, 2, 3)'

# LIKE operator (pro řetězce)
'"NAZEV" LIKE \'Pra%\'' # začíná na "Pra"
'"NAZEV" LIKE \'%ice\'' # končí na "ice"
```

6.6.2 Použití v kurzoru

```
# Jednoduchý dotaz
where = '"POCET_OB" > 1000000'
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV"], where) as tab:
    for r in tab:
        print(r[0])

# Složený dotaz
where = '"KATEGORIE" IN (1, 2) AND "POCET_OB" > 100000'
with arcpy.da.SearchCursor("Sidla.shp", ["NAZEV"], where) as tab:
    for r in tab:
        print(r[0])
```

6.7 Časté vzory použití

6.7.1 Export prvků podle kategorie

```
kategorie = set([r[0] for r in arcpy.da.SearchCursor("Sidla.shp", ["KATEGORIE"])]))

for kat in kategorie:
    where = f'"KATEGORIE" = {kat}'
    vystup = f"Sidla_kat_{kat}.shp"
    arcpy.Select_analysis("Sidla.shp", vystup, where)
```

6.7.2 Vytvoření slovníku z tabulky

```

kraje_dict = {}

with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as tab:
    for r in tab:
        kraje_dict[r[0]] = r[1]

print(kraje_dict)
# {'Praha': 1280508, 'Středočeský': 1338925, ...}

```

6.7.3 Hledání maxima s atributy

```

max_ob = float('-inf')
max_kraj = {}

with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB", "SHAPE@AREA"]) as tab:
    for r in tab:
        if r[1] > max_ob:
            max_ob = r[1]
            max_kraj = {
                'nazev': r[0],
                'obyvatel': r[1],
                'plocha': r[2] / 1000000 # km2
            }

print(f"Největší kraj: {max_kraj['nazev']}")
print(f"Obyvatel: {max_kraj['obyvatel']}")
print(f"Plocha: {max_kraj['plocha']:.0f} km2")

```

6.7.4 Procházení s indexem

```

i = 0
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV"]) as tab:
    for r in tab:
        i += 1
        print(f"{i}. {r[0]}")

```

6.8 Tipy a triky

6.8.1 Rychlé získání počtu prvků

```
pocet = 0
with arcpy.da.SearchCursor("Kraje.shp", ["OID@"]) as tab:
    for r in tab:
        pocet += 1
print(f"Počet prvků: {pocet}")

# Alternativa pomocí GetCount
pocet = int(arcpy.management.GetCount("Kraje.shp")[0])
```

6.8.2 Práce s None hodnotami

```
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POZNAMKA"]) as tab:
    for r in tab:
        nazev = r[0]
        poznamka = r[1] if r[1] is not None else "Bez poznámky"
        print(f"{nazev}: {poznamka}")
```

6.8.3 Kombinace více WHERE podmínek

```
# Pomocí proměnných
min_ob = 500000
max_ob = 1500000
where = f'"POCET_OB" >= {min_ob} AND "POCET_OB" <= {max_ob}'

with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV"], where) as tab:
    for r in tab:
        print(r[0])
```

6.8.4 Zpracování s chybovým hlášením


```
try:
    with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as tab:
        for r in tab:
            print(f"{r[0]}: {r[1]}")
except Exception as e:
    print(f"Chyba při zpracování: {e}")
```